Reverse Engineering a Vulnerable Android Application

Analysis, Modification and Security Reporting

Student	Manahil Saleem – 74408 Tooba Zahid - 63535
Course	SSDD theory (Assignment 1)
Department / Program	BSCB
Project repository	**https://github.com/ManahilSaleem02/Reverse-engineering-project**

1. Introduction and Scope

This report documents a controlled security assessment of DIVA (Damn Insecure and Vulnerable App), a deliberately vulnerable Android application provided for training. The work was done in the role of a junior mobile application security analyst. Only the instructor-provided training application was analysed, on a personally owned test device, and no third-party software was examined or modified. The goals were to understand the application's structure, identify a security weakness, and make a controlled modification that shows how application resources and code determine runtime behaviour.

2. Application Details and Environment

Application	DIVA (Damn Insecure and Vulnerable App)
Package	jakhar.aseem.diva
Original file	diva-beta.apk
Version	versionName 1.0, versionCode 1 (from AndroidManifest.xml)
SDK levels	Original: minSdkVersion 15, targetSdkVersion 23. Rebuilt APK reports target 24 (aapt dump badging).
Tools	JADX-GUI, Apktool 3.0.3, ADB, zipalign, apksigner, JDK keytool, Git/GitHub
Test device	Physical Android phone connected via USB (no emulator)

3. Task 1: Setup and Original Behaviour

The original APK was installed on the physical device using ADB and launched. The main menu lists the challenge modules (for example Insecure Logging and Hardcoding Issues). In the Insecure Logging module the user enters a credit card number and taps Check Out; the application always reports a generic error and gives no indication of what was recorded internally.

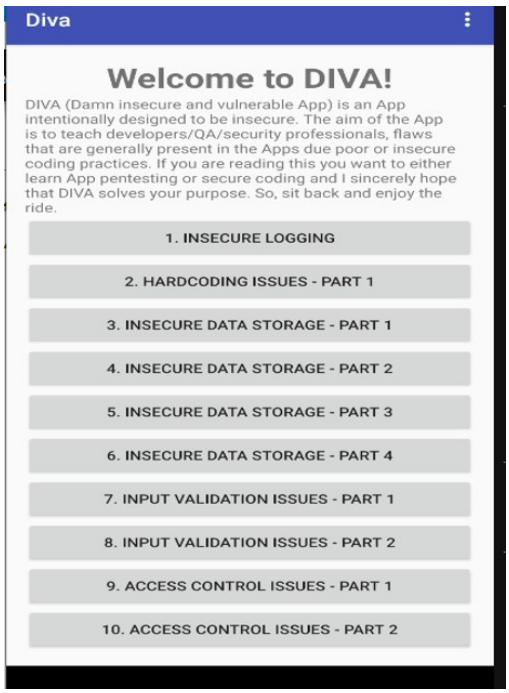


Figure 1: original DIVA main menu before modification

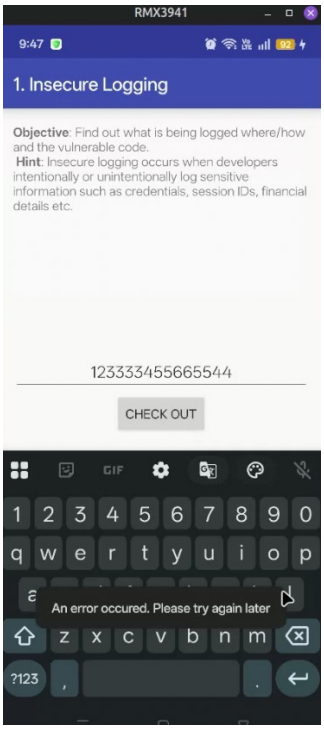


Figure 2: Insecure Logging screen with a test card number entered; only a generic error toast is shown

4. Task 2: Reverse Engineering

The APK was opened in JADX-GUI. The AndroidManifest.xml was reviewed for the package name, requested permissions, declared activities, exported components and the debuggable and backup flags. One feature, Insecure Logging, was traced end to end: the layout provides a text field and a button bound to the checkout() method of jakhar.aseem.diva.LogActivity; checkout() reads the entered text and passes it to processCC(), which deliberately fails; the catch block then handles the failure.

Key manifest findings: package jakhar.aseem.diva, version 1.0 (code 1), minSdkVersion 15 and targetSdkVersion 23; permissions WRITE_EXTERNAL_STORAGE, READ_EXTERNAL_STORAGE and INTERNET; and the application flags android:debuggable="true" and android:allowBackup="true". LogActivity is declared as one of the challenge activities and takes its label from the string resource d1.

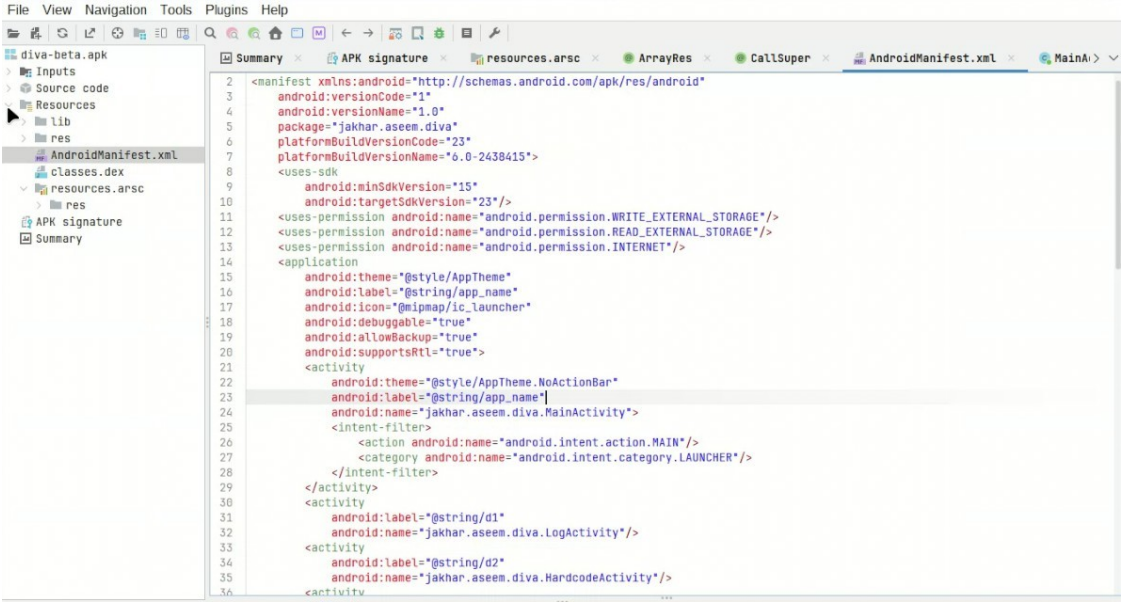


Figure 3: AndroidManifest.xml in JADX-GUI

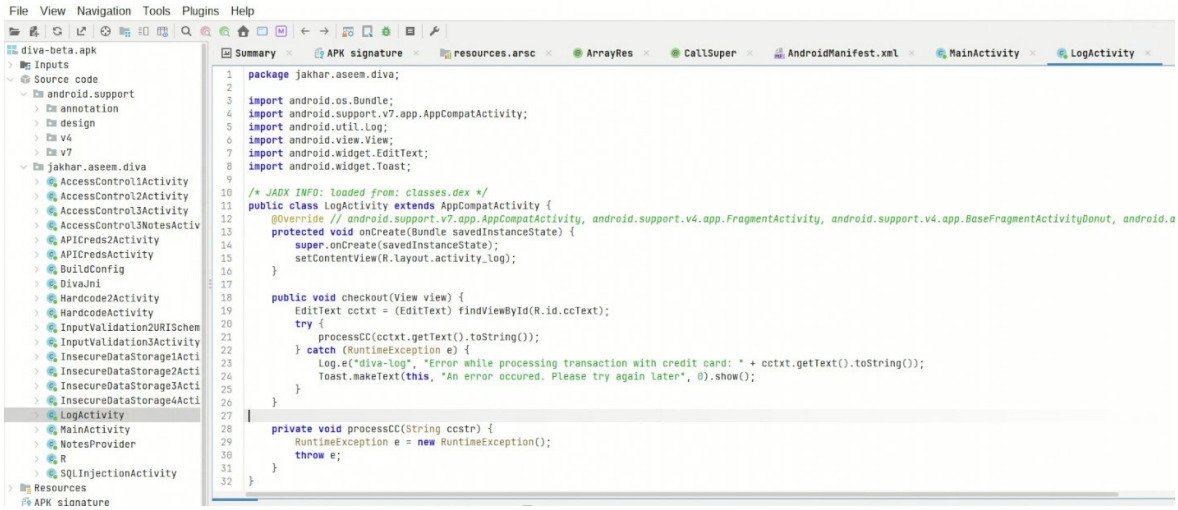


Figure 4: LogActivity.checkout() and processCC() in JADX-GUI (Log.e line 23)

5. Task 3: Vulnerability Identification and Evaluation

Insecure logging of sensitive data (CWE-532; OWASP MASVS-STORAGE). In LogActivity.checkout(), the failure handler writes the full credit card number entered by the user to the Android system log using Log.e with the tag diva-log, while the user only sees a generic error toast.

Location	jakhar.aseem.diva.LogActivity, method checkout(View)

Impact	The card number can be read from logcat through ADB, and on older Android versions by other apps holding the READ_LOGS permission. Logs are also frequently collected in bug reports and diagnostics, so the data can persist beyond the user's session.
Likelihood	Moderate on modern devices (needs ADB or physical access), higher on legacy devices, lost or shared devices, and when logs are exported.
Related observations	The manifest also sets android:debuggable="true" and android:allowBackup="true", which would further weaken a production build.
Risk rating	Medium to High, because the exposed data is regulated financial information.
Remediation	Never log user-supplied sensitive values. Log only identifiers or error codes, mask values if unavoidable, and remove debug logging from release builds (R8/ProGuard rules or BuildConfig.DEBUG checks).

The generic toast message gives users a false sense of safety: the interface hides the failure details while the log discloses the secret. This is an evaluation of risk, not just a description, since the cost of introducing the flaw (one debug statement) is tiny compared with the severity of its consequences.

Dynamic confirmation. Running adb logcat -s diva-log while pressing Check Out showed each entered card number in plain text in the log, confirming the finding from static analysis.

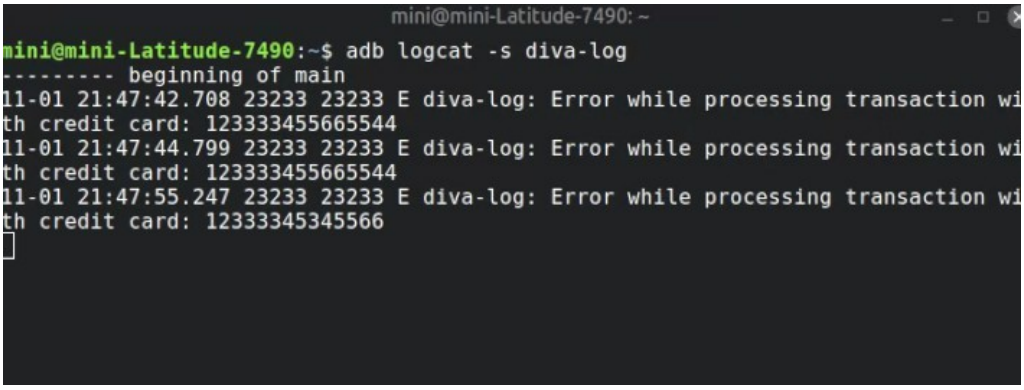


Figure 5: logcat output exposing the entered credit card numbers

6. Task 4: Controlled Modification

The approved modification changed a visible label to demonstrate the link between resources and behaviour. The APK was decoded with Apktool, the main menu string in res/values/strings.xml was changed to "1. Hacked by Manahil Saleem", and the package was rebuilt with Apktool 3.0.3, aligned with zipalign, and signed with apksigner using a self-generated key (RSA 2048-bit, SHA384withRSA, 10,000 day validity, CN=manahil, C=PK).

6.1 Source change



Figure 6: git diff of res/values/strings.xml showing the label change (d1)

6.2 Signing key generation

```
Generating 2,048 bit RSA key pair and self-signed certificate (SHA384withRSA) wi
th a validity of 10,000 days
        for: CN=manahil, C=PK
[Storing my-release-key.keystore]
```

Figure 7: keytool generating the self-signed signing key

6.3 Rebuild, align and sign

```
mini@mini-Latitude-7490:~/MANAHIL 1/uni material/5th sem/SSDD theory/assignment1$ java -jar apktool_3.0.3.jar b decoded -o diva-re
built-v2.apk
I: Using Apktool 3.0.3 on diva-beta.apk with 8 threads
I: smali has not changed.
I: Building resources with aapt2...
I: Building apk file...
I: Importing lib...
I: Built apk into: diva-rebuilt-v2.apk
mini@mini-Latitude-7490:~/MANAHIL 1/uni material/5th sem/SSDD theory/assignment1$ zipalign -f 4 diva-rebuilt-v2.apk diva-aligned-v
2.apk
mini@mini-Latitude-7490:~/MANAHIL 1/uni material/5th sem/SSDD theory/assignment1$ apksigner sign \
--ks my-release-key.keystore \
--out diva-final-v2.apk \
diva-aligned-v2.apk
Keystore password for signer #1:
mini@mini-Latitude-7490:~/MANAHIL 1/uni material/5th sem/SSDD theory/assignment1$ aapt dump badging diva-final-v2.apk | grep -E "s
dkVersion|targetSdkVersion"
sdkVersion:'15'
targetSdkVersion:'24'
```

Figure 8: apktool build, zipalign, apksigner sign, and aapt badging check of the SDK levels

6.4 Signature verification

```
mini@mini-Latitude-7490:~/MANAHIL 1/uni material/5th sem/SSDD theory/assignment1
$ apksigner verify --verbose diva-final-submission.apk
Verifies
Verified using v1 scheme (JAR signing): true
Verified using v2 scheme (APK Signature Scheme v2): true
Verified using v3 scheme (APK Signature Scheme v3): true
Verified using v4 scheme (APK Signature Scheme v4): false
Verified for SourceStamp: false
Number of signers: 1
```

Figure 9: apksigner verify reports Verifies, with v1, v2 and v3 schemes true and one signer

The v4 scheme is false, which is expected because no v4 signature file was requested; it does not affect installation.

6.5 Result on the device



Figure 10: the modified application showing "1. HACKED BY MANAHIL SALEEM" on the main menu (the button style renders the text in capitals)

The first menu item now shows the new label while the rest of the application functions normally. Because the signing key differs from the original, the original app had to be uninstalled before the modified build could be installed. The patch changes only the label; the logging vulnerability described in Section 5 is unchanged.

7. Evaluation and Lessons Learned

The exercise shows that a client-side application is fully inspectable: its manifest, resources and logic can be decompiled, edited, rebuilt and re-signed with freely available tools. Security therefore cannot depend on the client keeping secrets or enforcing rules alone. Sensitive values must not be logged or stored in plain text, release builds should remove debug output, and important checks must be enforced server-side. Obfuscation (R8) and integrity checks raise the effort required but do not replace secure design. This supports SDG 9 by encouraging resilient, trustworthy software. Limitations: the assessment used one deliberately vulnerable app, mainly static analysis with a single dynamic check, and one device.

8. Version Control Evidence

All stages of the work (setup, analysis, decompilation, patching and final build) were committed chronologically to a public GitHub repository: **https://github.com/ManahilSaleem02/Reverse-engineering-project**. The commit history serves as proof of consistent work.

9. Conclusion

The DIVA application was analysed statically, and an insecure logging weakness that exposes credit card numbers in the system log was identified and evaluated as Medium to High risk. A controlled resource modification was applied, rebuilt, signed (verified with apksigner) and deployed to a physical device, confirming that application behaviour follows directly from its resources and code. The recommended fix is to remove sensitive data from logs and strip debug logging from release builds.

References

OWASP Mobile Application Security Testing Guide (MASTG) and MASVS; CWE-532: Insertion of Sensitive Information into Log File; Android Developers documentation on logging and security; Apktool and JADX project documentation; DIVA Android by Payatu / Aseem Jakhar.